

GVZork

As long as we've had computers, people have wanted to use them for playing games. Even Alan Turing - considered the father of computer science - wrote algorithms for how a computer could play chess. There wasn't yet a computer capable of running the algorithm, so he proved it worked by following the algorithm himself in a game. Many years later, a group of students and researchers at MIT created the text adventure game "Zork". Based on an earlier game called "Colossal Cave Adventure", Zork became a near instant hit, and a fondly remembered classic. The Internet Archive has Zork available to play at

https://archive.org/details/msdos_Zork_I_-_The_Great_Underground_Empire_1980

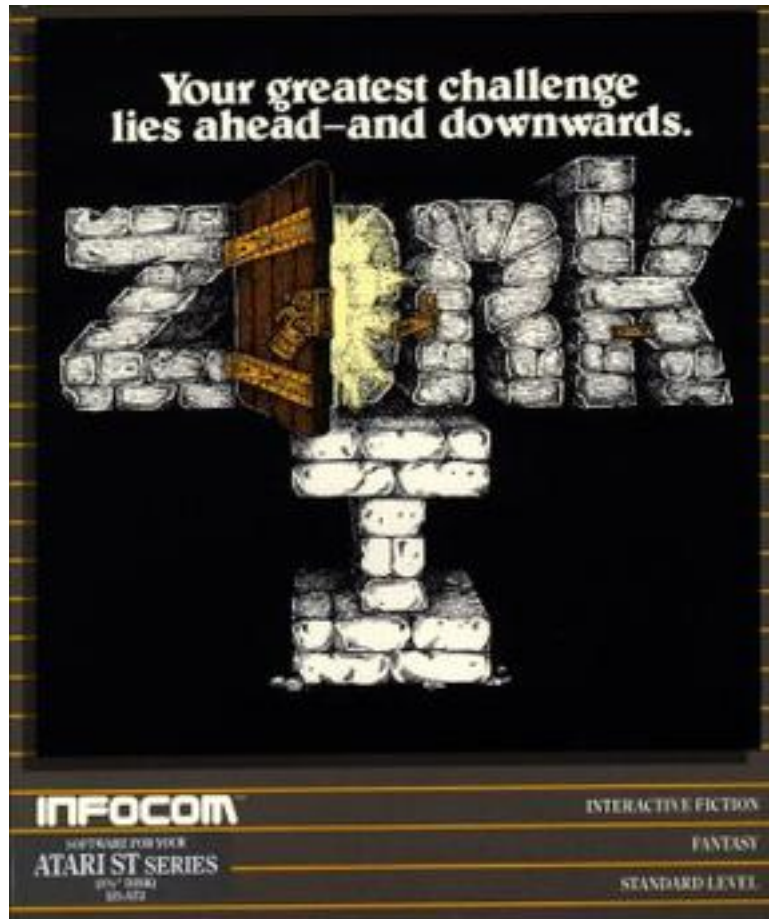


Figure 1: Zork I Box Image

Zork provides a world players can interact with via text based commands; for instance you might type look and the game would reply with West of

House. You are standing in an open field west of a white house, with a boarded front door. There is a small mailbox here. . The user might then enter, `open mailbox` to find a leaflet they can take, or might enter, `go east` to proceed to an area to the east (if one exists). Part of Zork's popularity came from the variety of commands, locations, and items the players encountered.

GVZork

We are going to create our own text-based adventure game, called GVZork. Our plot for GVZork is that someone has stolen Brutus the Bulldog's collar (Ferris' mascot). An evil troll that lives under Ferris' campus has put a spell on GV, freezing campus in time. You are an adventurer coming to campus to try and save us. To do this, you must roam campus to find food. You will take the food to an elf that lives in the ravines behind campus. If you give the elf enough food, it will break the spell. If you give it something it can't eat, it will transport you to a random location on campus.

The game world must have multiple locations (at least 10). A location will be connected to other locations via a unique direction; for instance if you are in "Mackinac Upstairs" you might be able to type `go downstairs` to go to "Mackinac Downstairs". Or, if you are in Padnos you might be able to type, `go west` to reach the Rec Center. Each location should have a name but also a description. For instance, Mackinac Upstairs might have the description, "The School of Computing and your professor's office is here."

Locations can have zero or more items. Each item has a name and a description. Some items are edible; these provide some number of calories. Items that aren't edible provide 0 calories. Once the elf has eaten enough calories it will reverse the spell. Items also have a weight, and the adventurer can only carry 30 pounds at once. The player can pick up and drop items at any time. A dropped item becomes part of the location's items.

One of the best parts of the original Zork game was its command system. Instead of a single word, some commands had multiple synonyms that could be used to perform the action. For instance, the user could enter `get`, `take`, or `grab` and the name of an item to add it to the inventory. This flexibility added to the game's appeal.

Our game should support the commands:

- `help`, `?` - prints the commands and a help message
- `look` - prints the current location description, a list of items in the location, and a list of directions you can go
- `go DIRECTION`, `leave DIRECTION` - moves the player in the specified direction, if the current location has a location in that direction
- `take ITEM`, `get ITEM`, `grab ITEM`, `pickup ITEM` - adds the specified item to the player's inventory, if the item exists in the current location and the

player has the weight capacity to carry it

- **give** ITEM, **drop** ITEM, **toss** ITEM - removes the specified item from the player's inventory (if it exists there), and adds it to the location's items
- **items**, **inventory** - shows the weight the player is carrying and a list of all items in inventory
- **quit**, **exit** - print a message and exit

Your command system should be flexible and easily extensible. You will need to create two custom commands (and may wish to create more for fun!).

Sample Run

Here is a sample of how the game should look:

Welcome to GVSU! Unfortunately, a student stole Brutus the Bulldog's collar from Ferris State. Not many people know this, but Ferris has a troll that lives under their campus. The troll cast a spell on GVSU, freezing the entire campus in time. There is one way to fix this though - in the woods behind campus there is a magical elf who loves food. He is known to be very picky, but if you feed him enough food he will save campus. If you feed him something he doesn't like, he will transport you far away and you must try again.

Please dear hero - find the food the elf desires and save our campus!

Valid commands are:

- help
- look
- go
- give
- take
- items
- quit

What is your command (type 'help' for instructions)? look

You location is: Padnos Hall - Lots of science labs are in this building.

You see:

- Rusty_Nail - 1 lb - A rusty nail (I hope you've had a tetanus shot).
- Beaten_Anvil - 60 lb - An anvil that looks as if it has been abused.

From here you can go:

- North

What is your command (type 'help' for instructions)?> take Beaten_Anvil

You can't carry that much weight.

What is your command (type 'help' for instructions)?> take Rusty_Nail

```
You have taken the Rusty_Nail - 1 lb - A rusty nail (I hope you've had a tetanus shot).
What is your command (type 'help' for instructions)?> go North
You are now in Manitou Hall Downstairs - There are several classrooms and a drink machine.
```

```
What is your command (type 'help' for instructions)?> items
In your inventory you carry 1 pounds:
```

```
- Rusty_Nail - 1 lb - A rusty nail (I hope you've had a tetanus shot).
```

```
What is your command (type 'help' for instructions)?>
```

Our design makes use of `Item`, `Location`, `NPC`, and `Game` classes. We will describe each of these below. If you wish to see a demonstration of how the game should work, you can view one at <https://youtu.be/MbX2PmOgYEI>.

Unfortunately, your boss was rather ‘old-school’, and gave you sample code that uses getters and setters instead of properties. You need to update it to use properties. This **does not mean** that you should change any of the method names given to you. Simply that you should have read and write properties for all fields specified in your class, and you should modify any code in the `Game` class to use them.

Why are we giving you “broken” code? A few reasons: first, this is a very real-world example. You often get old code and need to update it. This involves a time consuming process of finding where the old code was used and updating it accordingly. Second, to further reinforce our learning on getters/setters and properties. Third, it requires you to read through the code and - more importantly - to read and decipher error messages. This is designed to be a challenge, and I ask that you **not** use AI to do this for you. It could make these changes in a few moments, but then you don’t get the practice reading code and error messages **that will be your most essential skills in the workforce.**

All instance variables for a class must be created in the constructor; declaring them elsewhere is poor practice and will result in a loss of points.

Item

An `Item` represents objects the player may encounter along the way, and is an object that has a `name: str`, a `description: str`, the number of `calories: int` it can provide (0 if it isn’t edible), and a `weight: int`. It must provide:

- A constructor that can take a `name`, `description`, `calories`, and `weight`.
- You must ensure the following:
 - the `name` variable cannot be blank. Raise a `ValueError` with an appropriate message if a blank string is passed in.

- the `calorie` variable cannot be less than 0 or more than 1000 and must be an `int`.
- the `description` cannot be blank.
- the `weight` must be an `int` and must be between 0 and 500.
- A `__str__` method that returns a string representing the `Item`. The format should be
NAME - X lb - DESCRIPTION, i.e. Rusty Nail - 1 lb - A rusty nail (I hope you've had a tetanus shot)

You can ensure that a variable is of the desired type with the `isinstance()` function (<https://docs.python.org/3/library/functions.html#isinstance>). If, for instance, you wanted to make sure `length` was a floating point number, you could type

```
if isinstance(length, float):
    ... do something ...
```

NPC

The `NPC` class represents information about a character that can be in a `Location`. An `NPC` has a `name: str`, `description: str`, `message_number: int`, and list of `messages: list[str]`. Each time the player speaks to an `NPC`, the message number should increase by 1, resulting in the next message in the list being printed the next time the player speaks to the character. The message number should go back to 0 after it goes past the length of the message list. The class must have:

- a constructor that accepts a `name` and a `description`
- a method `get_message()` that returns the current message (as indicated by message number), then changes the message number appropriately
- a `__str__(self) -> str` method that returns only the name of the `NPC`

Location

Each `Location` represents a place on campus that may be visited. A `Location` holds a `name: str`, a `description: str`, a `bool` that indicates if it has been yet visited, a dictionary of `neighbors`, a list of `NPCs`, and a list of items. The `neighbors` dictionary will have a key that is a direction, that refers to a `Location` as its value. For instance, if we have a `Location` called `zumberg` and another `Location` called `kirkhoff`, `zumberg` would have a dictionary entry “west” that holds `kirkhoff`. We then want `kirkhoff` to hold in its dictionary a key-value pair of “east” that refers to `zumberg`. If we want to have two locations attached such that the player can enter but not exit the way they came, we can merely leave the entry off one of the objects. For instance, we could have a `portal Location` that has a “through” key pointing to `zumberg` but no corresponding key-value pair in the `zumberg` neighbor dictionary.

Location must implement:

- a constructor that takes a name and description
- a `get_locations` function that returns the neighbors dictionary
- an `add_location(self, direction: str, location: 'Location')` `-> None` method. This function will add the location into the dictionary with the provided direction string. If the string is blank, raise a `ValueError`. If the key already exists in the dictionary, raise a `KeyError`.
- an `add_npc(self, npc: NPC)` method for adding an NPC to the Location's list, and a `get_npcs(self) -> List[NPC]` function that returns the list of NPCs.
- an `add_item(self, item: Item)` method for adding an Item to the Location's NPC list, and a `get_items(self) -> List[Item]` method to return the Items.
- a `set_visited(self) -> None` method that changes the visited variable to True. Once a location is visited, it can no longer be False. Also, include a `get_visited(self) -> bool` function for checking if the location has been visited.
- a `__str__` method that returns the name and description formatted as
NAME - DESCRIPTION, i.e. Padnos Hall - Lots of science labs
are in this building.

Game

The game takes place in a world of connected locations. The purpose of the game is to collect edible items and then bring them to the elf in the woods behind campus. The elf needs 500 calories of edible food before it will save GVSU. Each item can have 0 or more calories. If a player gives the elf something inedible (something with 0 calories), the elf will be displeased and will teleport the player to a random location in the world. The player can only carry 30 pounds at a time, so multiple trips to the elf may be needed. Once the elf has enough calories, it will save campus and the game will end.

The `Game` class holds all the logic for a game instance. This class will hold several pieces of data. Make sure it has

- a dictionary of commands
- a list of Items the player currently has in their inventory
- an int that holds the current weight the player is carrying
- a list of Locations that exist in the world
- a variable to hold the player's current location
- an int to hold the number of calories the elf needs before it will save the campus
- a bool to store whether the game is still in progress

Game will adhere to the **Command Pattern**. Instead of having a loop body with a large number of `if` statements, for instance

```
command = input("What is your command? ")
if command == 'help':
    self._help()
elif command == 'talk':
    self._talk()
elif command == 'go':
    self.move()

... On and on ...

elif command == 'quit':
    self.quit()
```

We can instead create a data structure that maps commands to functions. Python makes this simple with dictionaries. We could for instance do something like this:

```
self._commands = { 'help': self.show_help, 'talk': self.talk, ...
→ and so on ..., 'quit': self.quit }
```

In this example, the values of the dictionary (i.e. `self._show_help`, `self.talk`, and `self.quit`) are the *names* of functions. Note that we are not *calling* the functions by putting the parentheses after the name (we didn't type `self.quit()` for example); we are merely giving the name of the method. Now, our input loop can look something like

```
command = input("What is your command? ")
self._commands[command]()
```

Notice here that we *did* use the parentheses to call the function. We can do this because Python supports **first-class function**. This simply means that a function can be used just like any other piece of data. For instance, we can store a function in a list or dictionary (as we did here), or we could pass a function as a parameter to another function.

The Command Pattern allows us to abstract away all of the `if` statements. We merely call the function that corresponds with the typed command (if one exists). Note that we can check if the command is a key of our dictionary with the `in` keyword:

```
if input in self._commands:
    ... do something ...
```

The `Game` class will require the most code to be written. It needs

- a constructor that takes no parameters (other than `self`). The constructor will set the commands dictionary equal to the return call from our `setup_commands` function. It will call the `create_world` method. It will

then set default values for all other variables. Set the current location to a random location selected from the `random_location` method.

- a `create_world(self)` -> `None` method that creates all the `Locations`, `Items`, and `NPCs` in the game. This function can get messy, as it will have a lot of text for names and descriptions of objects. Remember that your code should not be more than 72 characters long (this is marked in PyCharm with a vertical line). If a line goes beyond that, you should separate it into more than one line. You can do this by splitting the string:

```
kirkhoff_upstairs = Location("kirkhoff upstairs", "The  
→ student union. There are restaurants, a store, and  
→ places to congregate.")
```

becomes

```
kirkhoff_upstairs = Location("kirkhoff upstairs", "The  
→ student union. There are restaurants, a store, and "  
                           "places to congregate.")
```

This implicitly concatenates the two strings together, so formatting this way poses no issues.

In this function you will need to add all `Items` and `NPCs` to the rooms in which they belong, as well as add each `Location` to the neighbors to which it needs to connect. Also, be sure to add each `Location` to the `self._locations` list. Because there will be so much setup code, you may wish to break the function into commented regions, wherein each region focuses on the creation and setup of a single `Location`. Your game needs at least 8 `Locations`. There is no requirement for how many `Items` and `NPCs` must be in each `Location`, but your game does need at least 10 `Items` and 5 `NPCs`.

- a `setup_commands(self)` -> `Dict[str, Callable]` method. This method will create a new dictionary. The keys will be a command such as `talk`, `give`, `go`, etc. The values of the dictionary will be the names of the functions that should be called for each of those commands. Note that we can have more than one command per function; for instance, we could have both “?” and “help” correspond to a `self.show_help` method. Be sure this function returns the dictionary.
- a `random_location(self)` -> `Location` method that selects a random `Location` from the locations list and returns that `Location`.
- a `play(self)` -> `None` method. This is the core game loop. It should first print a message describing the game, then call the method to list all commands. Then, while the game is still in progress, it will loop. In the loop, we will prompt the user for a command. The user may enter multiple words in a prompt. We will split the user’s input into a list of words. We can split on spaces with code `tokens = user_response.split()`.

Once we have the tokens list, create a variable called `command` and set it equal to the first element in the list. Then, remove the first element with the `del(tokens[0])` command. Then, use the code `target = ' '.join(tokens)` code to put the remaining tokens together. Thus, if the user enters

```
talk ball of light
```

then `command` will be equal to `talk`, and `target` will be `ball of light`

We will then call the function from the commands dictionary by using this key. Pass `target` as a parameter to the called function. If the command does not exist in the dictionary print a message to the user telling them so.

Once the loop ends (i.e. the in-progress variable is `False`), check if the elf got enough calories. If it did, print a success message and quit. Otherwise, print a failure message and quit.

- a `show_help(self)` method that prints out a help message and all the commands from the command dictionary's keys. This method must also post the current time. You will need to read the `datetime` documentation to do this - <https://docs.python.org/3/library/datetime.html>. Print the time in a nicely formatted string, but you can decide if you wish to use 12 or 24-hour time.
- a `talk(self, target) -> None` method. This method will check if the provided NPC is in the current room. If it is, it will call the NPC's `get_message` method and print the resulting message.
- a `meet(self, target) -> None` method. It will check if the NPC is in the room, and if it is will ask the NPC for its description and print it.
- a `take(self, target) -> None` method. If the target item is in the room it will remove it from the room's inventory, add it to the user's inventory, and add the weight of the item to the user's carried weight.
- a `give(self, target) -> None` method. Removes the target item (if it exists) from the user's inventory, adds it to the current location's inventory and decreases the user's carried weight. The function will then check if the current location is the woods. If it is, it will check if the item given was edible. If the item is edible, reduce the number of calories the item has from the total the elf needs. If the item was not edible, transport the player to a new location by setting the current location equal to the return from `random_location`.
- a `go(self, target) -> None` method. Sets the current location's visited status to `True`. Checks if the player has over 30 weight; prints a message and returns if so. Otherwise, it checks if provided direction exists in the current location's neighbor dictionary. If so, sets the current location equal to the value from the dictionary.

- a `show_items(self, args: str = None) -> None` method. This method doesn't need any parameters but has an `args` parameter so it can be called with the same syntax as the other commands. It should print all items the player is carrying, and the current amount of weight carried.
- a `look(self, args: str = None) -> None` method. This method doesn't need parameters either but has `args` for the same reason as given above. This method will print the current location, a list of `Items` in the location or a message indicating there are none, a list of `NPCs` in the room or the message "You are alone.", and a list of directions in which the player can go. If a location has been visited previously, print the direction and the location. Otherwise, simply print the direction.
- a `quit(self, args: str = None) -> None` method that prints a failure message and exits the game.
- two additional command functions that you design and create. You could add teleportation, magic, etc.

Submission

You may work on this project in groups of up to 3. While you may work on it individually or with a single partner, you are assuming the risk and responsibility of taking on the extra work. This project can be challenging as it makes use of multiple types of data structures, a design pattern you may not have encountered before, and is made up of several classes.

Submit only the Python file(s) that you created. **Do not submit a .zip file.** Zipped files cannot be marked up in Blackboard, meaning I have no easy way to give you feedback on your code.

Ensure that **every** file has all team members' names at the top in the docstring, as well as a date. If your name is not on the file, you will not receive a grade. By allowing your name to be placed in the file you are attesting that you worked on this file an adequate amount and that you understand **all** code in it. Failure to adhere to these criteria may be academic dishonesty and can result in penalties.

Grading

Your code will be graded on the following by the following rubric. You can receive a maximum of the listed percentage of points for each section:

- 10% Commented according to the style guide. Either the Python or Google style guides are accepted. Feel free to have AI generate comments, but be sure you validate them all before submission.
- 10% `Item` class works as specified
- 10% `NPC` class works as specified
- 10% `Location` class works as specified
- 10% Adheres to the Command Pattern

- 10% Includes at least 8 connected **Locations**, 10 **Items**, and 5 **NPCs**. You can add more, but no extra credit will be given
- 10% Each of the required commands and functions are complete and work correctly
- 10% Two extra commands and corresponding functions are created and implemented correctly
- 20% The game is playable as described